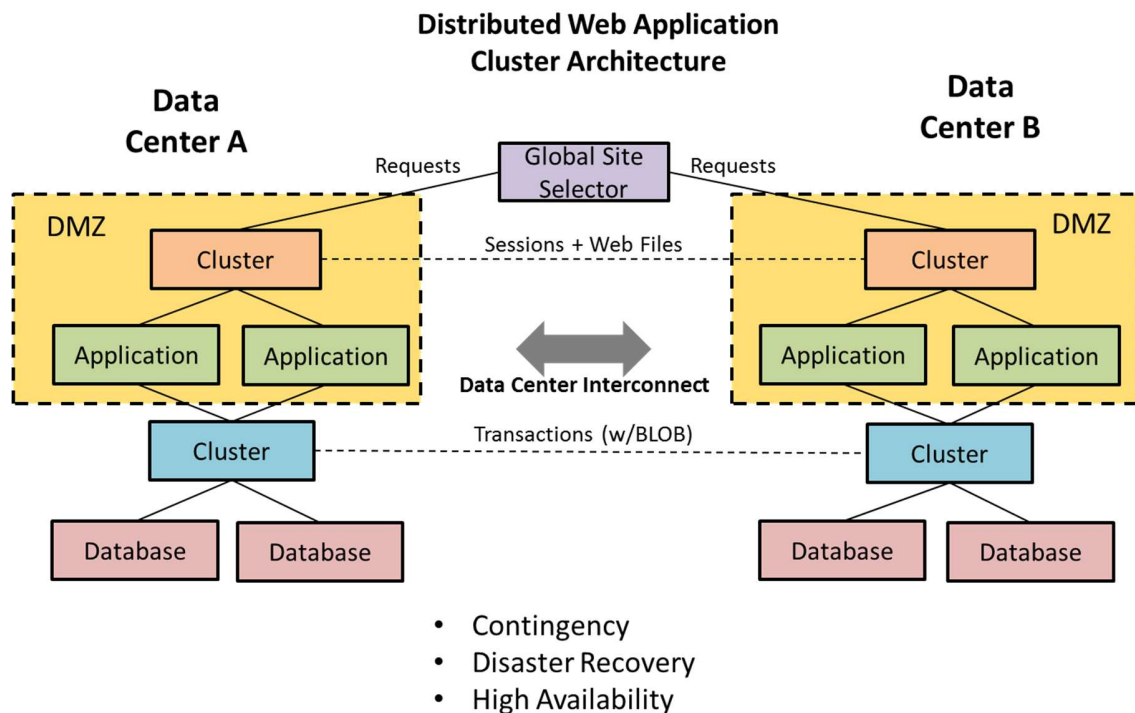


ARQUITECTURA GENERAL



Perspectiva General

En el diagrama anterior se muestra la arquitectura general de un clúster de aplicaciones Web distribuida, la cual constituye la base para la habilitación de aplicaciones, bases de datos, sistemas y servicios Web de misión crítica con operación continua en dos o más centros de datos geográficamente dispersos, con el propósito de lograr contingencia ante siniestros temporales, recuperación de desastres ante siniestros permanentes o alta disponibilidad.

En esta arquitectura tanto las aplicaciones como las bases de datos operan como un clúster, tanto en la red local (LAN) como en la red global (WAN) entre los centros de datos (clúster estirados), otorgando al usuario final disponibilidad y continuidad de los servicios Web que consume.

En cada clúster utilizan mecanismos especializados de distribución de la demanda (balanceo), tolerancia a fallos (conmutación) y redundancia (replicación/sincronización) de datos, la selección de estos mecanismos depende de la compatibilidad entre estos y el software/hardware con el cual están construidas las aplicaciones, bases de datos, sistemas y servicios Web. Así mismo, es importante complementar la selección con una política de respaldo que permita lograr niveles adecuados de recuperabilidad y continuidad de los servicios.

En realidad, no existe un estándar universal certificado para el diseño de un clúster de aplicaciones Web distribuidas, sin embargo, existen mejores prácticas, pruebas de concepto y experiencias aplicables a cada tecnología que deben considerarse al momento de construir una solución de este tipo.

Este documento describe cuáles son las consideraciones más importantes para la implementación de clústers de aplicaciones Web distribuidas que utilizan las siguientes tecnologías: sistema operativo Linux, servidor Web Apache, lenguaje de programación PHP, balanceador pgPool de instrucciones SQL, base de datos PostgreSQL, y opcionalmente, balanceador HAProxy de solicitudes Web. **Este documento en ningún caso cuantifica tecnología, define responsabilidad o acuerdo entre personas sobre actividad o proceso alguno relacionado con su contenido.**

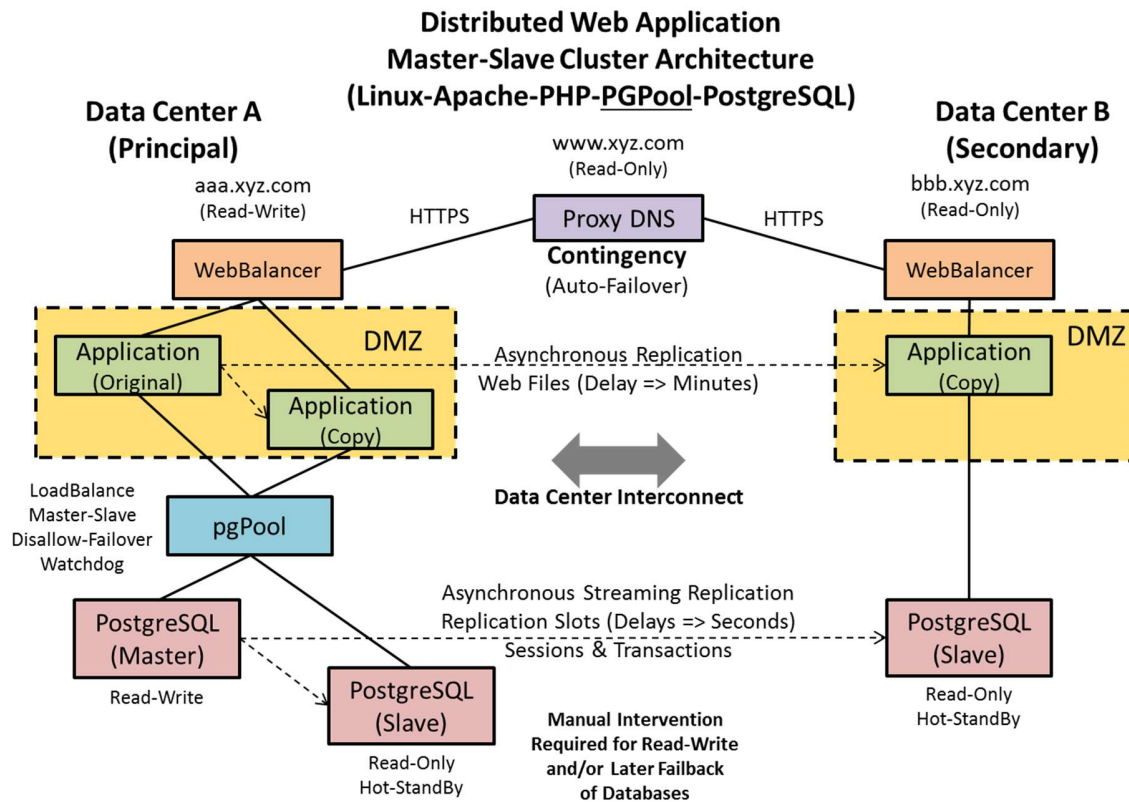
Consideraciones Generales

- **Versiones de software mínimas recomendadas:** Ubuntu Linux 16.04 LTS con Kernel 4.4 de 64 bits (otros sistemas operativos puede no estar suficientemente estables y con paquetes con versiones recientes), Apache 2.4, PHP 5.6/7.0, PostgreSQL 9.5, pgPool 3.6.1, y opcionalmente, HAProxy 1.7. En el caso de pgPool y HAProxy será necesario descargar y compilar el código fuente en lugar de utilizar las versiones incluidas en los repositorios del sistema operativo, debido a la cantidad de errores críticos de estabilidad corregidos, así como, las nuevas funcionalidades de alto valor incluidas hasta la fecha (Febrero/2017) en estas versiones.
- Reconocer que **no todas las aplicaciones** desarrolladas para utilizar el manejador de base de datos PostgreSQL, **han sido diseñadas y/o certificadas para funcionar con el balanceador pgPool** de instrucciones SQL, generalmente es un ejercicio de ensayo y error. PostgreSQL y pgPool son productos interoperables pero completamente independientes. Es importante destacar que pgPool surgió como una solución para la incapacidad de PostgreSQL de reutilizar una misma conexión para varios clientes (*Connection Pooling*) y fue progresivamente evolucionando hasta lograr las funcionalidades que actualmente posee. De la misma forma, existen también métodos complementarios al balanceo de consultas (SQL), tales como el uso de cachés, los cuáles se describen con más detalles en el apartado de “Application” en la sección de Arquitectura Específica.
- Reconocer que **no todas las aplicaciones** desarrolladas para utilizar el servidor de aplicaciones Web denominado Apache, **han sido diseñadas y/o certificadas para funcionar con el balanceador HAProxy** de solicitudes Web, generalmente es un ejercicio de ensayo y error. Apache y HAProxy son productos interoperables pero completamente independientes. De la misma forma, existen también métodos complementarios al balanceo de solicitudes Web, tales como el uso de aceleradores (i.e. APC) y cachés de solicitudes HTTP (e.g. varnish), útiles en caso de crecimiento exponencial de la demanda (Véase las referencias al final del documento).
- Reconocer por una parte, que pgPool no es la única herramienta que existe para lograr alta disponibilidad de base de datos, y por otra parte, **no hay actualmente una herramienta para PostgreSQL que ofrezca de forma integrada todas las funcionalidades esenciales para construir un clúster geo-distribuido** de base de datos, a saber: Connection Pooling, Load Balancing, Query Partitioning and Multi-Master Replication.
- **El uso de cualquier software similar a pgPool tiene impacto en el diseño de la arquitectura y codificación de las aplicaciones y bases de datos**, es necesario que los desarrolladores conozcan su arquitectura y funcionamiento en profundidad aun cuando no lo administren. **Existen diversas restricciones en pgPool (especialmente para ambientes geo-gráficamente dispersos) siendo más conveniente replicar las bases de datos utilizando el propio manejador de base de datos PostgreSQL (Streaming Replication)**, y dejando exclusivamente a pgPool, como gestor de conexiones (Connection Pooling) y balanceador de instrucciones (Load Balancer). Así por ejemplo, pgPool actualmente adolece de las siguientes restricciones:
 - Está diseñado para ser utilizado con todos los servidores ubicados en una misma red local.
 - Instrucciones SQL/DDI con valores autogenerados (SERIAL, SEQUENCE, ID, OID), valores aleatorios (RANDOM) o temporales (CURRENT DATE, NOW) pueden provocar inconsistencias en el modo de replicación.
 - Es incompatible con tablas temporales (volátiles).
 - Objetos binarios de gran tamaño (campos BLOB) puede producir un cuello de botella.
 - Clusterizarlo requiere al menos 3 instancias (servidores con pgPool).
 - No es posible su encadenación o conexión recursiva (pgPoolA ... pgPoolB), principalmente porque requiere interacción directa con el manejador de base de datos para coordinar las estrategias de sincronización.
 - No encola las transacciones ni realiza reintentos, simplemente excluye los servidores con fallas, hasta que sean re-sincronizados de forma automática (por él mismo) o manualmente por el administrador de base de datos.

- En caso de falla, puede ejecutar el cambio de roles y re-sincronización de servidores degradados pero no distingue entre servidores locales y remotos ni múltiples maestros. Esto implica que para ambientes geo-distribuidos entre dos o más centros de datos, el intercambio de roles debe ser realizado manualmente por el administrador de base de datos.
 - Para operar en el modo maestro/esclavo, la replicación debe ser implementada en el manejador de base de datos o software secundario.
- **El uso de cualquier software similar a HAProxy tiene impacto en el diseño de la arquitectura y codificación de las aplicaciones,** es necesario que los desarrolladores conozcan su arquitectura y funcionamiento en profundidad aun cuando no lo administren. Por ejemplo, las sesiones (PHP) de los usuarios deben ser almacenadas directamente en la base de datos para evitar la pérdida de la sesión de usuario cuando sus solicitudes Web sean balanceadas a servidores diferentes.
- **Ambos pgPool y HAProxy están diseñados para balancear (replicar) servidores ubicados en una misma red local** de alto desempeño, no para balancear (replicar) servidores geo-gráficamente dispersos.
- Los mecanismos utilizados para lograr alta disponibilidad, tales como: duplicidad/clúster local de servidores, balanceo de instrucciones SQL de base de datos y solicitudes Web, replicación de datos (archivos y transacciones) u otros, deben estar complementados por **políticas de respaldo locales e independientes en cada centro de datos** como requisito básico para poder recuperar la operación de los servicios después de una situación de contingencia.
- **No existen asistentes de configuración y mantenimiento de un clúster de aplicaciones Web distribuidas,** se requiere experticia en interfaz línea de comandos Linux, para completar la configuración, integración, y posteriormente, elaboración de scripts adicionales de control y gestión de la solución. **En tal sentido, es importante contar con los siguientes roles técnicos: arquitectos, desarrolladores y administradores especializados en redes y seguridad, sistemas operativos, aplicaciones y bases de datos, además de los líderes y gerentes responsables por el ciclo de vida de la solución.**
- Debido a la **ausencia de herramientas de respaldo en línea (sin interrupción de servicio) de bases de datos PostgreSQL** en el mercado, existen las siguientes alternativas para realizar respaldos:
 - **Tradicional (Recomendada):** mediante vaciados (DUMP) de base de datos en archivos ubicados en un espacio adicional en el disco local del servidor de base de datos.
 - **Ventajas:** no hay interrupción de servicio.
 - **Desventajas:** requiere duplicar la cantidad de espacio requerido para la base de datos. Requiere más tiempo para copiar los respaldo al servidor, y luego, reconstruir la base de datos (import/restore).
 - **Alternativa (No recomendada):** detener el manejador de base de datos, crear una instantánea (Snapshot) a nivel de sistema operativo, iniciar el manejador de base de datos, respaldar los datos y eliminar la instantánea (snapshot).
 - **Ventaja:** la interrupción de servicio es mínima y no se pierden transacciones (simplemente se demora la réplica momentáneamente). Al momento de una falla en la base de datos, bastará con copiar los respaldos al servidor e iniciar el manejador de base de datos, no será necesario realizar una reconstrucción de la base de datos (import/restore).
 - **Desventaja:** introduce un punto de falla en el proceso natural de gestión de la base de datos porque el respaldo puede fallar y la base de datos puede quedar detenida a la espera de

intervención manual. Esta opción es útil para respaldar servidores esclavos, especialmente cuando una interrupción de servicio no es una opción factible en el servidor maestro.

ARQUITECTURA ESPECÍFICA

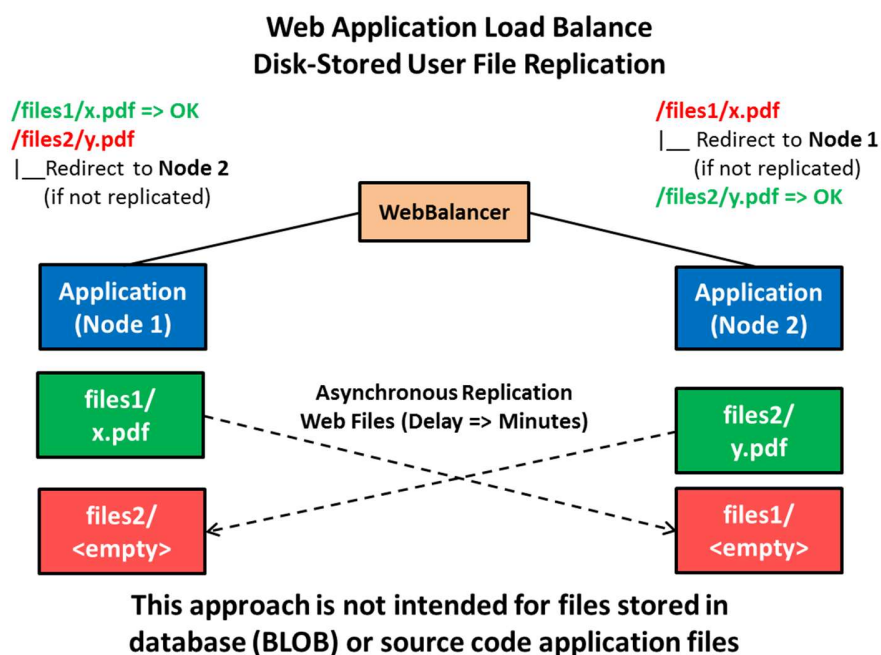


Consideraciones Específicas

- **Data Center A:** será considerado como el centro de datos principal de hospedaje de las aplicaciones, bases de datos, sistemas y servicios Web.
- **Data Center B:** será considerado como el centro de datos secundario de hospedaje de las aplicaciones, bases de datos, sistemas y servicios Web.
- **Proxy DNS:** se implementa con un enrutador o selector global de sitios (Global Server Load Balance) que permite modificar las respuestas a consultas de nombres de dominio (DNS) según la disponibilidad de los servidores de destino, redirigiendo de forma transparente para el usuario las solicitudes Web hacia los servidores de aplicaciones de ambos centros de datos simultáneamente, o bien, al centro de datos secundario en caso de falla de todos los servidores del centro de datos primario, y viceversa.
 - **www.xyz.com:** Las solicitudes hacia este dominio son dirigidas hacia los centros de datos A y B, no debe ser utilizado cuando se requieran realizar operaciones de escritura sobre las bases de datos (Write).
 - **aaa.xyz.com:** Las solicitudes hacia este dominio solo son dirigidas hacia el centro de datos A, sus bases de datos operan en modo de lectura y escritura (Master/Read-Write).
 - **bbb.xyz.com:** Las solicitudes hacia este dominio solo son dirigidas hacia el centro de datos B, sus bases de datos son réplicas de las bases de datos del centro de datos A y operan en modo solo lectura (Slave/Hot-StandBy/Read-Only). En caso de desastre en el centro de datos A, y mediante intervención manual del administrador de base de datos, pueden configurarse para operar en modo de lectura y escritura (Master/Read-Write).
 - La **lógica de control de flujo** codificada en las aplicaciones debe utilizar los sub-dominios "www", "aaa" y "bbb" del dominio "xyz.com" para garantizar la escritura y lectura correcta en las bases de datos en cada uno de los centros de datos. Por ejemplo, la carga de archivos en las

aplicaciones debe ser redirigida al sub-dominio “aaa”, de lo contrario generará un error si es ejecutada en el sub-dominio “bbb”.

- **WebBalancer:** se implementa en cortafuegos, enrutadores o balanceadores basados en hardware o software (e.g. HAProxy) en modo activo-pasivo o activo-activo hacia servidores Web.
- **DMZ:** Se implementa en cortafuegos (Firewall) basados en hardware o software, para proteger las bases de datos de daños colaterales derivados de una intrusión a los servidores de aplicaciones, está previsto colocar los servidores de aplicaciones en una zona (red) desmilitarizada (DMZ), la cual debe contar con soporte para detección de intrusos (IDS), bloqueo de intrusos (IPS) y protección ante ataques de denegación de servicio (DDoS) para todos los puertos de los servidores Web (HTTP/HTTPS). La administración remota debe realizarse (a través de enlace dedicado) hacia la red privada de los servidores para evitar la publicación de puertos sensibles (e.g. FTP, SSH) en Internet.
- **HTTPS:** Todos los servidores de aplicaciones deben estar protegidos con el uso de certificados de seguridad (SSL) para cifrar las comunicaciones de los usuarios. Dichos certificados deberán ser instalados directamente en los servidores de aplicaciones, y preferiblemente en equipos y/o componentes de seguridad (Firewall, IDS, IPS, DDoS) capaces de inspeccionar el tráfico y prevenir explotación de vulnerabilidades Web. Actualmente existen alternativas abiertas, libres y automatizadas la generación y renovación de certificados (e.g. *Let's Encrypt*, <https://www.letsencrypt.org>).
- **Data Center Interconnect:** es una arquitectura redundante de conexión entre centros de datos.



- **Application:** se implementa con servidores Linux, preferiblemente virtualizados. La replicación asíncrona unidireccional de archivos Web (Original -> Copy) se implementa a nivel de sistema operativo con el comando “rsync”, el cual es programado para ejecutarse y monitorearse periódicamente en un intervalo constante mayor o igual a 10 minutos, mediante la revisión del código de retorno y registro en la bitácora (syslog) de la tarea programada (cronjob). Por una parte, la replicación manual permite a los desarrolladores sincronizar los cambios en el código fuente de las aplicaciones entre todos los servidores, y por otra parte, la replicación automática sincroniza los

archivos cargados por los usuarios en los servidores Web que no son almacenados en la base de datos en campos binarios (BLOB). Como se muestra en el diagrama precedente, cuando los archivos no se almacenan a la base de datos y los servidores son sujetos a un balanceo de solicitudes Web, es necesario que la aplicación al momento de la carga del archivo registre en base de datos cuál servidor recibió el original y cuál fue el tamaño de éste, así el resto de los servidores puede redireccionar al al servidor que posee el original si aún no ha recibido una copia completa. Es importante que las consultas repetitivas hacia la base de datos sean almacenadas en una memoria cache (e.g. memcached), esto permite reducir la carga sobre los servidores de base datos y acelerar el tiempo de respuesta a los usuarios, sin embargo, esto requiere modificación de las aplicaciones para que utilicen la memoria cache, la cual generalmente se implementa en los mismos servidores de aplicaciones Web.

- **Application-Cluster:** La creación de clúster de alta disponibilidad con base en las herramientas (e.g. redhat-cluster-suite, heartbeat/pacemaker) del sistema operativo Linux para automatizar el inicio de los servidores de aplicaciones Web (Apache) y el reinicio del sistema operativo en caso de inhibición, es factible, opcional y recomendado.
- **pgPool:** el cliente debe garantizar que su aplicación Web sea compatible con pgPool, el cual en principio está previsto sea instalado en el mismo servidor de base de datos PostgreSQL. En caso contrario será necesario incluir servidores adicionales para hospedar pgPool, lo cual puede requerir cambios en la estructura de red. La interacción local de todos los pgPool en un mismo centro de datos se realizará a través de su módulo de coordinación (watchdog), encargado de publicar una dirección IP única para todos los servidores de aplicaciones y definir el servidor maestro de pgPool. Si la aplicación Web no es compatible con pgPool debe conectarse de forma directa sólo al servidor de base de datos PostgreSQL, como sigue: con el servidor que tenga rol de maestro de lectura y escritura (Master/Read-Write), y en su defecto, con el servidor que tenga rol de esclavo de solo lectura.
- **PostgreSQL:** se implementa con servidores Linux, preferiblemente virtualizados. La replicación entre servidores de base de datos es asíncrona por transmisión (Asynchronous Streaming Replication) desde el servidor de base de datos PostgreSQL con rol de maestro de lectura y escritura (Master/Read-Write) ubicado en el centro de datos A, hacia los servidores con roles de esclavo de solo lectura (Slave/Hot-StandBy/Read-Only) ubicados en el centro de datos A (replica local) y B (replicas remota). La replicación de las transacciones ocurre en tiempo real tan pronto como son procesadas por el servidor maestro, sin embargo, existe un tiempo de retraso (delay) variable estimado en segundos, dependiente de la cantidad de transacciones, ancho de banda destinado a la replicación entre centros de datos u otros factores.
- **PostgreSQL – pgClusters:** el manejador de base de datos PostgreSQL tiene la capacidad de crear “pgClusters” administrados de forma completamente independiente, a nivel de: instancias, procesos, puertos, archivos, usuarios, grupos, permisos), los cuales permiten maximizar el aprovechamiento de recursos, reducir costos y facilitar la administración de PostgreSQL, manteniendo la segmentación de las bases de datos. Por ejemplo, en lugar de tener dos servidores Linux para hospedar las base de datos de las aplicaciones A y B, se crea un clúster para cada una de las aplicaciones A y B en el mismo servidor Linux. En caso que no se utilice el concepto de “pgCluster”, se deberán utilizar servidores de base de datos adicionales.
- **Application - Cluster:** La creación de clúster de alta disponibilidad con base en las herramientas (e.g. redhat-cluster-suite, heartbeat/pacemaker) del sistema operativo Linux para automatizar el inicio de los servidores de base de datos PostgreSQL y el reinicio del sistema operativo en caso de inhibición,

aun siendo factible no es recomendado porque los mecanismos de protección (i.e. fencing) puede afectar severamente la integridad de la base de datos.

REFERENCIAS

- Apache Web Server
<https://httpd.apache.org/docs/2.4/>
- Best Practices of HA and Replication of PostgreSQL in Virtualized Environments
<http://es.slideshare.net/jkshah/py-pg-day2013harep>
- Geographically Distributed PostgreSQL
http://es.slideshare.net/mason_s/geographically-distributed-postgresql
- HAProxy – High Performace TCP/HTTP Load Balancer
<http://www.haproxy.org/>
- Caché Alternativo de PHP (APC)
<http://php.net/manual/es/book.apc.php>
- Varnish HTTP Caché
<https://www.varnish-cache.org/docs/5.1/users-guide/intro.html>
- PHP Documentation – Extensions – Sessions - Functions
<http://php.net/manual/es/function.session-set-save-handler.php>
- PostgreSQL - Clustering
<https://wiki.postgresql.org/wiki/Clustering>
- PostgreSQL – Replication, Clustering and Connection Pooling
https://wiki.postgresql.org/wiki/Replication,_Clustering,_and_Connection_Pooling
- PostgreSQL - High Availability, Load Balancing, and Replication - Comparison of Different Solutions
<https://www.postgresql.org/docs/current/static/different-replication-solutions.html>
- PostgreSQL - High Availability, Load Balancing, and Replication - Cascading Replication
<https://www.postgresql.org/docs/devel/static/warm-standby.html>
- PostgreSQL - Hot Standby - Read Only Queries
https://wiki.postgresql.org/wiki/Hot_Standby
- PostgreSQL Replication - Second Edition - 2015 - PACKT Publishing Open Source*
https://books.google.co.ve/books?id=RmpGCgAAQBAJ&pg=PP7&lpg=PP7&dq=nasa+contributes+to+postgresql&source=bl&ots=RdR64N1bXj&sig=XAprbhPeABmy2806Rt1iohvSf5E&hl=es-419&sa=X&ved=0ahUKEwjo1sex6JzSAhVV_mMKHsw6DU04ChDoAQgZMAE#v=onepage&q=nasa%20contributes%20to%20postgresql&f=false
- pgPool Documentation – Restrictions
<http://www.pgpool.net/docs/latest/en/html/restrictions.html>
- pgPool Documentation – Online Recovery
<http://www.pgpool.net/docs/latest/en/html/runtime-online-recovery.html>
- pgPool Tutorial – Watchdog in Master – Slave Mode
http://www.pgpool.net/pgpool-web/contrib_docs/watchdog_master_slave/en.html
- Making master/slave system work better with pgPool-II (Commercial PostgreSQL Support)
www.sraoss.co.jp/event_seminar/2010/20100702-03char10.pdf
- Re: PostgreSQL HA config recommendations for several datacenters
<https://www.postgresql.org/message-id/CAEva%3DVnGHPM4MDS-GPTK0f1R1G7gf6FA6Yck1cVW0AQYLapqA@mail.gmail.com>
- PostgreSQL Replication and Load Balancing with pgpool-II
<http://pjkh.com/articles/postgresql-replication-and-load-balancing-with-pgpool2/>
- Yii Framework 1.1 – CdbHttpSession – Database as Session Storage
<http://www.yiiframework.com/doc/api/1.1/CdbHttpSession>
- Yii Framework 2.0 – yii\web\DbSession – Database as Session Storage

<http://www.yiiframework.com/doc-2.0/yii-web-dbsession.html>

- CMemCache implements a cache application component based on [memcached](#).
<http://www.yiiframework.com/doc/api/1.1/CMemCache>
- MemCache implements a cache application component based on [memcache](#) and [memcached](#).
<http://www.yiiframework.com/doc-2.0/yii-caching-memcache.html>